

Chapter 10

Reinforcement Learning

OBJECTIVE OF THE CHAPTER:

We discussed Supervised and Unsupervised learning in earlier chapters. The third type of learning is Reinforcement Learning (RL) where an AI agent learns from the consequences of its actions to perform a set of tasks within an environment. The popularity of RL is increasing rapidly within the AI community because of its usefulness in advanced AI applications like autonomous robots, self-driving cars, gaming, self-learning chatbots, and many more. The objective of this chapter is to provide an understanding of how RL works, its implementation, and its applications in modern industries.

10.1 INTRODUCTION

In previous chapters, we have discussed Supervised and Unsupervised learning in detail. In supervised learning, the agent is trained with labeled data. The labels are the correct actions a knowledgeable external supervisor identifies based on the training data that describes the situation. The objective is that the system will generalize or extrapolate the learning from these training data, and create a generalized ruleset so that the system can act correctly in situations outside the training data. However, the robustness of the rule is still dependent on the richness of the training data and the agent can't explore new actions outside the rule set. We can easily understand that in a highly interactive environment (for example a self-driven car), it is not practical to identify examples of

all the possible situations and train the agent on how to act in those situations. It is expected that the agent will explore and learn from its own experience. Reinforcement Learning addresses this area. As we have seen earlier, unsupervised learning also does not depend on labeled data, but its goal is very different - identifying the hidden pattern within the data.



10.2 WHAT IS REINFORCEMENT LEARNING (RL)?

Throughout our lives, the most significant portion of learning happens through interacting with our environment and gaining knowledge about the consequences of our actions in this environment. When we have a clear goal in mind, from these consequences or feedback, we can identify what is a right or a wrong action for a situation. For example, while conversing with someone, we constantly watch how our environment is responding and try to influence it to achieve our goal.

The computational approach to learning from interactions with the environment and analyzing the response to generate maximum reward (positive outcome) is the basis for Reinforcement Learning.

Reinforcement learning is a type of machine learning paradigm where the learner learns to map the actions to a situation and the goal is to maximize a numerical *reward* value. There is no expert to guide the learner on the right action (through labeled data), but it must discover which actions yield maximum reward value through *trial and error*. An interesting factor that impacts the learning path in RL is that the agent's current action may not only impact the immediate reward but may also impact the next action and as a result *all subsequent rewards*. These are very powerful learning concepts and mimic animal learning very closely. So, two key features behind all reinforcement learning algorithms are - **trial-and-error** and **delayed rewards or total cumulative rewards**.

The trial-and-error concept is closely related to the idea of *exploration* and *exploitation*. When the agent takes the known steps that have proven to be successful in yielding high reward values, then it is told to be *exploiting* the knowledge. On the other hand, when the agent tries out unknown paths and by doing that identifies the new ways to achieve higher rewards, it is termed as *exploration*. It is evident that without exploration the agent can't gain experience or have a probability to improve from the current max reward. But to do so, it may go through multiple failure paths. The balancing act between exploitation and exploration actions is one of the intensively studied areas of mathematics.

Points to Ponder:

- As there is no labeled data to train the agent, some may argue that reinforcement learning is a type of unsupervised learning, but there are significant differences. The reinforcement learning agent can benefit from learning the pattern in the data, but the agent's objective is not fulfilled by only that. It aims to maximize the reward signal within the environment.

In the subsequent chapters, we will discuss all these concepts in detail.

10.3 ELEMENTS OF REINFORCEMENT LEARNING

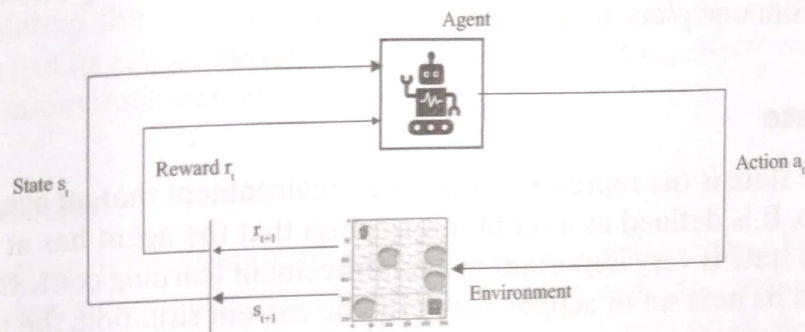


FIG. 10.1
Agent-Environment Interaction

10.3.1 Agent

An agent in the RL domain is the one who makes decisions on what action to take in an environment and in a particular state. It is the learner in RL. In the real world, it can be compared with the abstract decision-making unit like the CPU of a robot which makes the decision based on all the input from the robot's environment and applies the rules to take the best action for the situation.

10.3.2 Environment

Everything that the agent interacts with is called an environment. The environment can comprise anything that is outside the agent. In RL it can be demonstrated by the problem to be solved. Based on the properties of the environment, it can be classified as –

- *Deterministic vs. Stochastic environments* – an environment is called deterministic if the outcome is known based on the current state and action. For example, in a chess game, it is clear what will be the new state if a piece is moved. So, it is a deterministic environment. But if the outcome is not certain and only a probability of the possible outcomes can be estimated, then the environment is called a stochastic environment. For example, it is not certain what will be the outcome if a dice is rolled.
- *Fully observable vs. Partially observable environment* – when the state of the system is visible to the agent at all times, then it is called a fully observable environment. For example, in chess, the players can see the position of all the pieces on the board and thus it is a fully observable environment. But if the agent can't determine the state of the whole system all the time, then it is a partially observable environment.

For example, in a game of poker, the players can't see the cards in other players' hands.

- *Discrete vs. Continuous environments* – within an environment, if only a finite set of actions is possible for moving from one state to another, then it is called a discrete environment. For example, the number of possible actions is limited in chess. On the other hand, if the set of actions available is infinite, then it is called a continuous environment. For example, there can be infinite possible routes for traveling from one place to another.

10.3.3 State

The state is the representation of the environment that an agent is in a specific time step. It is defined as a set of information that the agent has at a given point in time. The state is very important in a reinforcement learning context as the agent needs to plan its next set of actions based on the current situation, the reward it can get from the environment, and the goal.

10.3.4 Policy

Policy is a strategy that the agent uses to achieve the goal. So, a policy defines the mapping from the perceived states of the environment to actions to be taken when in those states. Policy is a core component of reinforcement learning as it alone can drive the action of the agent. It can be deterministic, meaning that the environment state combination can provide the action information or it can be stochastic, meaning that the combination will return only the probabilities of different actions. In its simplest form, a policy π is a function that takes a state s as input and returns an action a . That is:

$$\pi(s) \rightarrow a$$

In this way, a policy is typically used by the agent to decide what action 'a' should be executed when it is in a given state 's'.

If the policy is stochastic instead of deterministic, then instead of returning a unique action 'a', the policy returns a probability distribution over a set of actions.

$$\pi(s) \rightarrow p(a)$$

For a policy-based reinforcement learning algorithm, the aim is to learn an optimal policy that achieves a specific goal.

10.3.5 Reward

After each action step, the environment sends a single number to the agent, which is termed a reward. These rewards define the goal of the reinforcement learning program because reinforcement learning algorithms have the goal of maximizing the rewards that can be accumulated by the agent over the long run. The effect of an action can be positive or negative, which is equivalent to reward and punishment in the real world. Through multiple trial-and-error the agent must learn to accumulate as much reward as possible and to do so it may have to alter the policy. Thus, reward becomes the key factor while creating or fine-tuning a policy. For example, if an action

by a policy yields a low reward, then the agent may choose to take a different action in a similar situation in the future.

If we take these entities together, then the agent and environment interact at a sequence of discrete time steps, $t = 0, 1, 2, 3, \dots$. At each time step t , the agent receives some information of the environment's state, $s_t \rightarrow S$, where S is the set of possible states, and on that basis selects an action, $a_t \rightarrow A(s_t)$, where $A(s_t)$ is the set of actions available in that state. As a result of the action, it will receive a reward r_t and keep on accumulating the rewards for all state-action pairs. After one time step, as a consequence of its action, the agent moves to a new state, s_{t+1} . Refer to Figure 10.1 to see the agent-environment interaction.

10.3.6 Value function

A value function defines long-term gain while reward defines immediate gain. When an agent takes an action in the current state, it can expect a certain reward. The action also affects the subsequent state's actions. The sum of all the rewards that can be accumulated by the agent starting from the current state to all the probable future states is termed as the value of the current state. In the real world, we can bring an analogy of pleasure or pain to reward, which is immediate effect, and long-term happiness or sadness to value function which is the accumulation of continuous pleasure or pain over a timeline. While it is a relatively straightforward task to calculate the immediate reward as it is given back by the environment directly, optimization of the value function is a complex problem to solve. For example, a particular step action may yield low immediate reward, but because its subsequent steps are yielding high rewards, the total accumulated reward over all probable future actions - the value of the step can be actually high, or vice versa. The value must be estimated and re-estimated over the entire lifespan of the agent based on the sequence of actions that the agent makes. While discussing different reinforcement learning algorithms, we will note that a key consideration of those algorithms is how to maximize the value function most efficiently.

10.3.7 Model

A model is a replica of the environment that imitates the behavior of the environment. This helps in deriving inferences about future courses of action based on prior knowledge about how the environment will behave in such a situation. So, an agent can identify the reward of a state and that of all probable future states without actually experiencing those situations. Models are well suited for planning as the future course of action can be decided based on the knowledge gained from the model of the environment. The technique of solving reinforcement learning problems using models is called the *model-based* approach and the main aim of the agent is to explore the environment to learn the model. On the other hand, the techniques that depend on exploration and exploitation of the environment to gather knowledge about it and identify the maximum value or optimal policy is termed a *model-free* approach.

10.3.8 Introduction to the Farama Foundation's 'The Frozen Lake' RL Environment

Let us take a famous gaming problem of *The Frozen Lake*. This game involves crossing a frozen lake by walking over it to reach the goal without falling into one of the holes. Refer to Figure 10.2. The lake is divided into a 4x4 grid world. The game starts with the agent placed at the location [1,1] and the goal is located at the far end of [4,4]. Holes are located at [2,2], [4,2], [4,3] and [1,4] locations. The player makes moves until it reaches the goal or it falls into a hole.

There are 4 different discrete actions possible in the action space – 0. Move Up, 1. Move Down, 2. Move Left, 3. Move Right.

The observation space has 16 discrete numbers. It is defined as the player's current position as $(\text{current_row} - 1) * \text{no_of_rows} + (\text{current_column} - 1)$. For example, the position of the goal is represented by $3 * 4 + 3 = 15$.

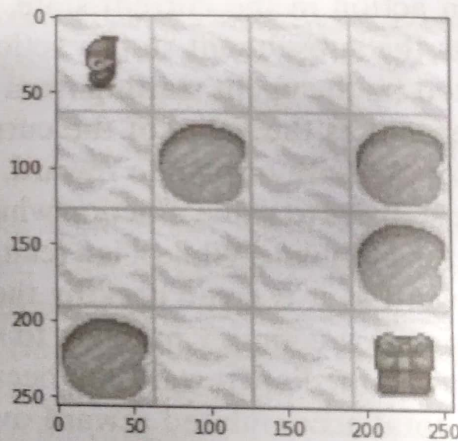


FIG. 10.2
The Frozen Lake Environment

More documentation about this environment can be found at https://gymnasium.farama.org/environments/toy_text/frozen_lake/

In Figure 10.3 different components of the Frozen Lake problem are explained.

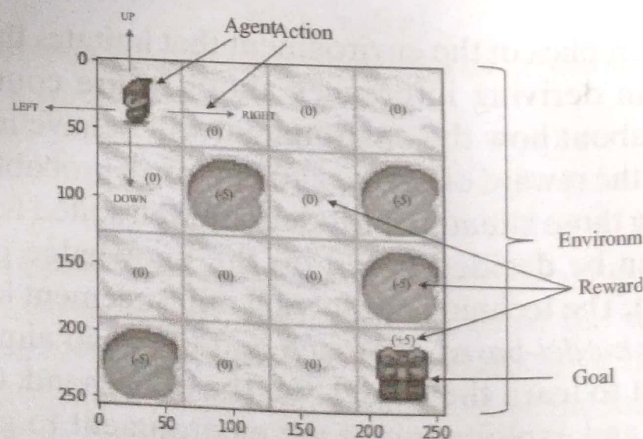


FIG. 10.3
Different elements of RL in The Frozen Lake environment

The **state** of the agent corresponds to its location on the lake grid, in this case, $s_t = (x, y)$ and $s_1 = (1, 1)$.

The **action** space, in this example, consists of four possible behaviors: move up, move down, move left, and move right.

The probability matrix P contains all pairwise combinations of states (s, s') for all actions in A . It's Bernoulli-distributed, and looks like this:

$$P_{\text{down}}((1,1), (1,2)) = 1; P_{\text{down}}((1,1), (1,3)) = 0; \dots; P_{\text{right}}((3,4), (4,4)) = 1$$

We can define the **reward function** R as (this is different than how Gymnasium defined it) –

- If it's in an empty cell, the agent receives a reward of 0, which means it is not earning an immediate reward so that it can look for accumulating rewards in the future
- If the agent is in a cell with a hole, it then receives a reward of -5 , which acts like a punishment and the expectation is that the agent will avoid this action in the future after learning
- If the agent successfully reaches the goal cell: $(4, 4)$ then it will earn a reward of $+5$
- The simulation runs for an arbitrary finite number of time steps but terminates early if the agent reaches a hole or the final goal cell.

Let's consider two policies π_1 and π_2 . We can indicate a policy as a sequence of actions starting from the state of the agent at $(1,1)$:

- π_1 : down $\rightarrow$ down $\rightarrow$ down: Hole
- π_2 : right $\rightarrow$ right $\rightarrow$ down $\rightarrow$ down $\rightarrow$ down $\rightarrow$ right: Goal achieved

By computing the value function, we get:

- $R(\pi_1): 0 + 0 - 5 = -5$
- $R(\pi_2): 0 + 0 + 0 + 0 + 0 + 5 = +5$

The evaluation of the policies suggests that the value is maximized with π_2 , which the agent chooses as its policy for this task. Refer to Figure 10.4

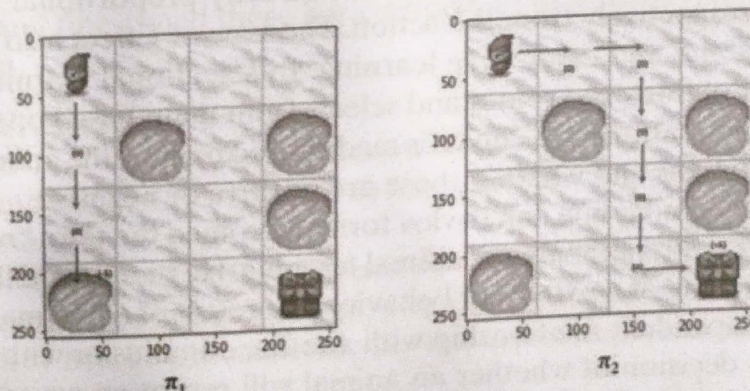


FIG. 10.4
Action – Policy – Reward concept

There are three distinct approaches to implementing RL algorithms which we can understand based on the discussion about value, policy, and model:

1. Value-based:

In the value-based approach, the objective of the agent is to find the optimal value function, which can maximize the value at a state under any policy. So, the agent looks for long-term returns at any state(s) under the policy π . The idea is that an optimal value function leads to optimal policy. By minimizing the loss between the predicted and the target value, the approximate true action-value function is arrived at and we get the policy directly from the value function.

2. Policy-based:

The second approach is to find the optimal policy for the maximum future rewards without using the value function. Here the policy is parameterized and the objective is to maximize the performance of that policy so that the action performed in each step helps to maximize the future reward.

The policy-based approach uses two types of policies:

- ❖ Deterministic policy: the policy (π) outputs the same action (A) at any state.
- ❖ Stochastic policy: instead of a particular action, the policy outputs the probability distribution of the actions.

3. Model-based:

A model-based approach is the third type, where a virtual model is created for the environment. The objective of the agent is to explore the environment to learn the model.

10.4 REINFORCEMENT LEARNING PRINCIPLES

Research around learning, especially on how to reinforce learning is not new. More than 100 years back in 1911, Edward Thorndike introduced the concept of the “Law of Effect”. In his paper, he described that an animal will repeat an action if the action produces satisfaction and will avoid an action if it causes discomfort. Moreover, the degree of pursuance or deterrence is directly proportional to the level of pleasure or pain respectively from the action. The Law of Effect also combined the concepts of *selectional* and *associative* learning. In selectional learning, the animal tries out different options and routes and selects from them based on the outcome of each. In associative learning, the choice is made based on how the animal associates its options with situations and whether those are positive or negative.

On this similar line, Ian Pavlov formally coined the term “*reinforcement*” in 1927 in the context strengthening of animal learning. He mentioned that reinforcement is the strengthening of a pattern of behavior as a result of an animal receiving a stimulus in a time-dependent relationship with another stimulus or with a response. This means that the decision of whether an animal will repeat an action in the future or not is affected by the reaction they have received shortly after they have taken the action. These concepts built the fundamentals of *trial-and-error* learning.

The idea of implementing the concepts in computer programming came out in 1948 when Alan Turing described in his report the design of a 'pleasure-pain system' that worked along the lines of the Law of Effect.

"When a configuration is reached for which, the action is undetermined, a random choice for the missing data is made and the appropriate entry is made in the description, tentatively, and is applied. When a pain stimulus occurs, all tentative entries are canceled, and when a pleasure stimulus occurs, they are all made permanent." - Turing, 1948

A second line of research that was going on at that time was related to "optimal control" and its solutions using value functions and dynamic programming. The idea was to design a controller that can maximize or minimize the measure of a dynamic system's behavior over time. Richard Bellman proposed an approach in the mid-1950s in which he suggested that a functional equation can be formulated using the dynamic system's state and a value function – "optimal return function" and the equation is known as the *Bellman equation*. Dynamic programming is the category of methods that solve optimal control problems by solving this equation. Bellman also proposed a discrete stochastic version of the optimal control problem, known as *Markov Decision Processes (MDPs)*. Though dynamic programming and MDPs were not related to any learning problem at that time, these became an essential part of solving RL problems as they provided a way to gradually reach correct answers through successive approximations, which was closely related to any learning problem.

A third, though less distinctive line of research that contributed to the development of modern reinforcement learning was temporal-difference learning. In 1959 Arthur Samuel proposed the implementation of a learning method that included *temporal-difference* ideas. The principle works based on the differences between temporally successive estimates of the same quantity, for example, the probability of achieving a total reward over a continuous time state. Arthur Samuel's famous checkers-playing program proved the efficacy of this learning method.

All these streams of research started coming together in 1989 when Chris Watkins developed the Q-learning model by bringing temporal difference and optimal control streams. We will discuss in this chapter how different modern RL algorithms implemented and fine-tuned these basic principles to achieve remarkable success in the RL domain.

1. Markov Property

In section 10.3 we discussed different elements of RL – like the state, action, reward, and policy. Let's see how to formulate any RL problem mathematically using those elements.

An RL problem is said to have *Markov property* if the future state depends only on the present and not on the past. This is named after the Russian mathematician Andrey Markov. Mathematically this can be represented as -

$$P[S_{t+1} | S_t] = P[S_{t+1} | S_1, S_2, \dots, S_t]$$

where S_t denotes the current state of the agent and S_{t+1} denotes the next state.

As we can see in the LHS of the equation, the transition from state S_t to S_{t+1} is entirely independent of the past and thus the RHS of the equation which has all the history from states S_1 to S_t gets captured within the current state S_t , if the process has Markov property.

Let's remember the definition of *Transition Probability* in this context. It is the probability that the agent will move from one state to another state. Drawing the similarity to moving from Markov State S_t to S_{t+1} i.e. from the current state to any other successor state, the state transition probability and the transition matrix are given by

$$P_{ss'} = P[S_{t+1} = s' | S_t = s] = \begin{matrix} & \begin{matrix} \text{"To State"} \\ \begin{bmatrix} p_{11} & p_{12} & \dots & p_{1n} \\ p_{21} & p_{22} & \dots & p_{2n} \\ \dots & \dots & \dots & \dots \\ p_{n1} & p_{n2} & \dots & p_{nm} \end{bmatrix} \end{matrix} \\ \begin{matrix} \text{"From State"} \\ \end{matrix} \end{matrix}$$

Here p_{12} represents the state transition probability of moving from state S_1 to state S_2 and so on. It should be noted that the sum of probabilities along the rows is always 1. This means that if we add up the probabilities of moving from one current state to multiple other future states, then the sum of such probabilities always needs to be 1. So,

$$p_{11} + p_{12} + p_{13} + \dots + p_{1n} = 1$$

2. Markov Process or Markov Chain

Now that we discussed state transition probability, let's understand how random processes are mathematically represented as Markov Chains or Markov Processes. A random process is a sequence of random states ($S_1, S_2, \dots, S_n$) that demonstrate Markov property. Thus, the environment can be represented using the states (S) and state transition probability matrix ($P_{ss'}$). In a discrete environment, it is called a Markov chain and in a continuous environment, it is termed a Markov process. Refer to Figure 10.5. In this example, we have 3 fully observable states A, B, and C. The probability of transitioning from one state to another or staying in the same state is provided above the arrows.

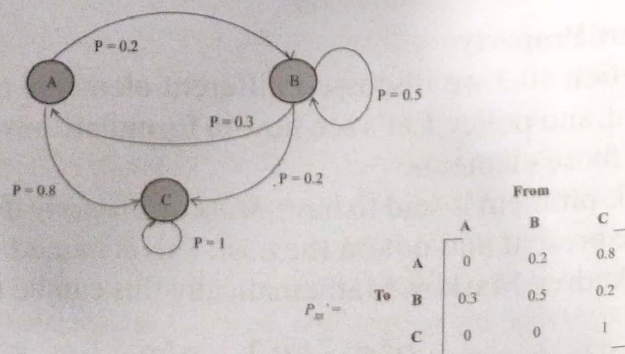


FIG. 10.5
Markov chain and process

We can create several random sets of states by sequencing the states like, A – B – C – C, A – B – B – C, and so on. The probability of being in a state S_{t+m} at a time step $t + m$ starting from state S_t is mathematically represented as,

$$p_{t+n} = p_t P_{ss'}^n, \text{ where } n \text{ is the number of steps between } S_t \text{ and } S_{t+m}.$$

When $n \rightarrow \infty$ (steady state), then the above equation can be generalized as,

$$p = p P_{ss'}$$

We can get an insight into which state of the Markov chain is visited in the long run by solving this equation. This is called the Chapman-Kolmogorov equation.

3. Markov Reward Process

Let's add the concept of reward as a result of moving from one state to another. A reward function evaluates how useful it is to transition from one state to another. This will guide the agent to choose which transition is good and which is not. Before we get into the Markov Reward Process (MRP), we should understand the terms – reward, return, and discount factor well.

Rewards are the numerical values an agent receives based on the action taken while being in a state in the environment. The reward can be positive or negative based on the action taken by the agent.

Return is the cumulative reward that the agent receives from the environment while completing a sequence of state transitions, called episodes. In RL the goal is to maximize the cumulative reward or return, rather than the reward an agent receives from the current state. So,

Return = $G_t = R_{t+1} + R_{t+2} + \dots + R_T$, where R_{t+1} is the reward received at time step $t = 0$, to R_T which is the reward received at the final time step T

Points to Ponder:

- **Episodic Task** – these tasks have a terminal or end state and contain a finite set of states. For example, in the frozen lake environment, the agent starts from the initial state, moves along the path and the game ends either if the agent falls into the pond or reaches the destination. Once the game is restarted, the agent will start from the initial state. So, every episode is independent.
- **Continuous Task** – the tasks that do not have a terminal state and thus never end are called continuous tasks. For example, learning a language is a continuous task. Calculating the returns of continuous tasks is tricky as they have an infinite set of states.

Discount factor (γ) is the value that determines how much importance is given to the current and future rewards. The return equation using the discount factor becomes

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \dots = \sum_{n=0}^{\infty} \gamma^n R_{t+n+1} \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} \dots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned} \quad (10.1)$$

So, if we define $G_T = 0$, or the return from the terminal step as zero, then this recursive equation works for all $t < T$.

The discount factor has a value between 0 and 1. So, if we choose a value closer to zero, then the future rewards get less significance, as the factor γ^n reduces the future values drastically. On the other hand, if we keep its value closer to 1, then all the future rewards will matter for calculating the return. Balancing the discount factor value is an important task in RL problems. So, there are two important aspects handled by γ —

- It handles the uncertainty of the future, as immediate rewards are valued more than the future rewards
- It helps to avoid infinite returns problem in continuous tasks as γ^n becomes very small as the value of n increases

In our earlier Markov process, we considered only the state transition and its probability. When a reward function is added on the Markov process then it represents a finite Markov Reward Process. The finite MRP can be represented by a tuple of $(S, P_{ss'}, R, \gamma)$,

Where S = finite set of states

$P_{ss'}$ = state transition probability

R = reward function $R_{ss'} = E[R_{t+1} | S_t = s, S_{t+1} = s']$, the expectation of the reward that the agent will get for moving from states s to s' in time t to $t+1$

γ = discount factor $0 \leq \gamma \leq 1$

in our earlier Figure 10.5 if we add rewards then the diagram will look like Figure 10.6

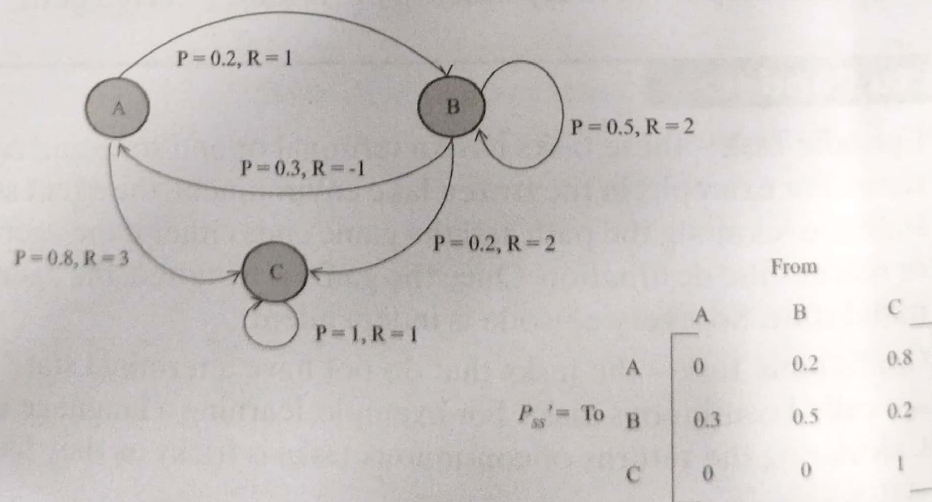


FIG. 10.6

Markov reward process

From the definition of the *expected value* of a random variable, which is the value of the variable multiplied by its probability, we get the expected value of transitioning from state A, $E[R^A]$ as $\sum r p(r)$, the value of reward associated with the state change multiplied by the transition probability of the state change. In our example in Figure

10.6, state A can change to state B or state C with rewards and probabilities (1, 0.2) and (3, 0.8) respectively. So,

$$\mathbb{E}[R^A] = 1 \times 0.2 + 3 \times 0.8 = 2.6.$$

This is the foundation for the value function and Markov Decision Process.

4. Bellman Equation for Value Function

The value function of a state determines how good it is for an agent to be in a particular state. The agent must take some action to find out the value or goodness of a particular state and thus the concept of policy comes into play here. As we discussed previously, a policy defines what actions to be taken in a particular state, s . So, the *state-value function* for policy π can be defined as,

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{n=0}^{\infty} \gamma^n R_{t+n+1} | S_t = s \right], \text{ for all } s \in S \quad (10.2)$$

where, $v_{\pi}(s)$ or value of the state, s under the *policy* is the expected return the agent can get starting from state, s to subsequent states, following a policy till the terminal state. *The value of the terminal state is defined to be zero.*

Similarly, the *state-action-value function* for policy π can be defined as,

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{n=0}^{\infty} \gamma^n R_{t+n+1} | S_t = s, A_t = a \right] \quad (10.3)$$

where $q_{\pi}(s, a)$ is the value of taking action a , in state, s under a policy π and is defined as the expected return starting from s , taking action a and following the policy π thereafter.

Based on Figure 10.6, if we take an episode with starting state A and terminal state C and which stays in state B in-between like, A – B – B – C, then the expected return with a discount factor of 0.5 is:

$$G_t = 1 + 2 * 0.5 + 2 * (0.5)^2 = 2.5$$

So, the value of State A is 2.5 for following the policy A – B – B – C.

Bellman's equation helps in finding out the optimal policies and value functions to maximize the return. But the complexity is that the reward is not a deterministic variable, but it is a random variable and thus return G_t also becomes a random variable. So, we can't maximize G_t directly, but we must maximize the expected returns, which we expressed in equations (10.2) and (10.3). From the above example, we can understand that the policy will change with experience and thus the value function will also change for different policies. Bellman equation aims to identify the optimal value function that can maximize the value compared to all other value functions.

Did you know?

- The Bellman equation is named after Dr. Richard Bellman who is called the Father of Dynamic Programming. In his paper "Theory of Dynamic Programming" in 1954, he explained with examples the principles of dynamic programming algorithms try to solve RL problems. In very simple terms, dynamic programming algorithms try to solve complex problems by breaking those down into simple sub-problems and solving the sub-problems recursively, meaning with functions that call themselves.

Also, recall from our discussion in section 10.3 that a Policy $\pi(a|s)$ is the probability of selecting an action, a if we are in state, s at time, t .

$$\pi(a|s) = \text{Probability}(A_t = a, S_t = s)$$

The fundamental property that is used for RL and dynamic programming is that they satisfy the recursive property, similar to what we have already established for the return (equation 10.1). So, equations (10.2) and (10.3) satisfy the below relationship between the value of state, s and its successor states, s'

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}_{\pi}[G_t | S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} | S_t = s] \end{aligned} \quad (10.4)$$

$$\begin{aligned} &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s']] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')] \text{ for all } s \in S \end{aligned} \quad (10.5)$$

It is important to note that the value function of the current state, s is now related to the value function of the successor state, s' , which enables us to iteratively compute the value function. This equation is called *Bellman equation for the state value function* and is one of the most important equations in RL. Similarly, the state-action function can be expressed as the recursive function with its successor state-action pair. In the next sections, we will discuss the approaches to solving this equation to identify the optimal value function.

A finite Markov Decision Process (MDP) is a mathematical model where the agent makes some decision or takes action based on its current state following a policy. It is a tuple of $(S, A, P_{ss'}, R(s|a), \gamma)$ where:

- S is a set of states,
- A is the set of actions the agent can take,
- $P_{ss'}$ is the transition probability matrix,
- $R(s|a)$ is the reward function,
- γ is the discount factor.

MDPs are a classical formalization of sequential decision-making problems where actions not only influence the immediate reward, but also the successor states and through those the future rewards. *The goal of RL is to find an optimal policy that will achieve a high value of cumulative reward over the long run.* Keeping this goal in mind, a policy π is considered better than or equal to policy π' if its return is better than or equal to that of π' for all states. Mathematically,

$$\pi \geq \pi' \text{ if and only if } v_{\pi}(s) \geq v_{\pi'}(s) \text{ for all } s \in S$$

So, there is always at least one policy that is better than or equal to all other policies, and this policy is called *optimal policy*. Optimal policy(s) are denoted by the following notation:

$$v_{*}(s) = \max_{\pi} v_{\pi}(s), \text{ for all } s \in S \quad (10.6)$$

Optimal policies share the same optimal action-value function, denoted as below:

$$q_{*}(s, a) = \max_{\pi} q_{\pi}(s, a), \text{ for all } s \in S \quad (10.7)$$

Here, for the state-action pair (s, a) , this function gives the expected return by taking action, a in state, s , and then following the optimal policy. So, q_* can be written in terms of v_* as:

$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a] \quad (10.8)$$

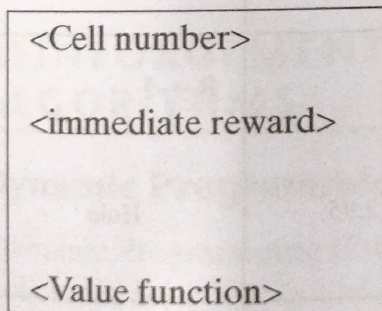
In equation (10.6), v_* is the optimal value function and thus it should satisfy the consistency condition that we discussed in Bellman's equation for state-value in equation (10.4). But as this is optimal value function, its consistency condition can be written without reference to any specific policy. This is the *Bellman optimality equation for v_** . Thus, we can deduce that the value of a state under an optimal policy must be equal to the expected return for the best action from that state:

$$\begin{aligned} v_*(s) &= \max_{a \in A(s)} q_{\pi^*}(s, a) \\ &= \max_a \mathbb{E}_{\pi^*}[G_t | S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi^*}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v_*(s')] \end{aligned} \quad (10.9)$$

Similar to equation (10.9), which is the Bellman Optimality equation for v_* , we can deduce the *Bellman Optimality equation for q_** as,

$$\begin{aligned} q_*(s) &= \mathbb{E} [R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a'} q_*(s', a')] \end{aligned} \quad (10.10)$$

Now that we have understood how the Bellman equation works, we can try to find out the value functions for the Frozen Lake environment with the parameters set in section 10.3.8. We will represent the environment as a grid world and the notations used are as below:



To calculate the value function of a state (cell here), we need the immediate reward of the state, s , and the value function of the next state, s' . We will use the simplified optimality function for state-value:

$$V(s) = \max [R(s) + \gamma V(s')]$$

Iteration 1:

Let's assume $\gamma = 0.9$ for our case. We will start with the calculation of the value function from the state immediately before the Goal state as we know that $V_{16} = 0$, as this is a terminal state. One of the episodes is $S_1 - S_5 - S_9 - S_{10} - S_{14} - S_{15} - S_{16}$

$$V_{15} = \max (R_{15} + 0.9 * (V_{16})) = 0 + 0.9 * (+5) = 4.5$$

$$V_{14} = \max (R_{14} + 0.9*(V_{15})) = 0 + 0.9*(4.5) = 4.05$$

$V_{13} = 0$ as this is a terminal state

As V_{13} is a terminal step, an alternative path to the goal should be explored. Let's assume that the path is V_9, V_{10}, V_{14} and we will use the next state-value functions accordingly.

$$V_{10} = \max (R_{10} + 0.9*(V_{14})) = 0 + 0.9*(4.05) = 3.64$$

$$V_9 = \max (R_9 + 0.9*(V_{10})) = 0 + 0.9*(3.64) = 3.29$$

$$V_5 = \max (R_5 + 0.9*(V_9)) = 0 + 0.9*(3.29) = 2.95$$

$$V_1 = \max (R_1 + 0.9*(V_5)) = 0 + 0.9*(2.95) = 2.66$$

In the same way, we can calculate the value function for other cells.

$$V_{11} = \max (R_{11} + 0.9*(V_{10})) = 0 + 0.9*(3.64) = 3.29$$

$$V_7 = \max (R_7 + 0.9*(V_{11})) = 0 + 0.9*(3.29) = 2.95$$

$$V_3 = \max (R_3 + 0.9*(V_7)) = 0 + 0.9*(2.95) = 2.66$$

$$V_2 = \max (R_2 + 0.9*(V_3)) = 0 + 0.9*(2.66) = 2.39$$

1 R = 0 Agent, V = 2.66 ↓	2 R = 0 V = 2.39 →	3 R = 0 V = 2.66 ↓	4 R = 0
5 R = 0 V = 2.95 ↓	6 R = -5 Hole	7 R = 0 V = 2.95 ↓	8 R = -5 Hole
9 R = 0 → V = 3.29	10 R = 0 V = 3.64 ↓	11 R = 0 ← V = 3.29	12 R = -5 Hole
13 R = -5 Hole	14 R = 0 V = 4.05 →	15 R = 0 V = 4.5 →	16 R = +5 Goal

We can see that when the agent is in S_1 , it won't move to right to S_2 but will move down to S_5 as $V_5 > V_2$. In this same way, it tries to identify the optimal path or policy to reach the goal.

Iteration 2...n:

Note that the value functions derived in *Iteration 1* may not be the optimal solution for this grid world. The reason is, that many of the cells can be reached from multiple paths and we should identify which of those is most efficient, meaning generates the most return. For example, S_{11} can be reached from S_{10} as well as from S_7 and we should calculate V_{11} for both paths and take the max value for the optimal solution. So, for each terminal episode, the value functions are evaluated, and fine-tuned and finally, the optimal policy (set of actions) is chosen based on the one that provides the maximum cumulative value or return. This is the key concept of the Bellman equation and optimality.

It may seem an easy task to solve the optimality equations to learn the optimal policies. But in reality, the computational cost to generate the optimal policy is prohibitively high, especially when the lengths of the episodes are very large. Memory requirement is also a constraint as building up approximations of value functions, policies and models requires a large amount of memory. So, some sort of approximation strategy is required to balance the acceptable optimality level and the resource availability. Because of the online nature of RL problems, it is often possible to focus on frequently occurring states and learn the best policies for them, rather than putting effort into states that seldom occur. This is the key aspect to be kept in mind while solving MDPs for RL problems. We will discuss a few such algorithms to compute optimal policies in the MDP environment in the next section.

10.5 REINFORCEMENT LEARNING ALGORITHMS

10.5.1 Dynamic Programming (DP)

Dynamic Programming (DP) is a set of algorithms that are used to compute optimal policies in a perfect model of the environment, like MDP. The key idea is to search for the optimal policy by using the value functions where the problem is broken down into multiple sub-problems and the result of a sub-problem is used in similar sub-problems, instead of independently solving those. To be solved through a dynamic programming approach, a problem should have the following properties – a) we should be able to divide the problem into similar sub-problems, and b) if we can find the optimal solution for a sub-problem, then that solution can be used to find the optimal solution for the overall problem. As we have seen in the Bellman equation (10.4), MDPs possess these properties because the value of a state is equal to the immediate value plus the value of the next step. This demonstrates the suitability for applying DP. As the computational overhead of a classical DP method is very high, most of the algorithms attempt to achieve almost the same effect as DP with

less computational cost and without assuming the perfect environment, though the fundamental theory of DP is followed.

In Dynamic Programming, the key concept is to first identify the value function that will satisfy the Bellman equation and then using the optimal value function identify the optimal policy. The process is logically divided into 3 steps –

1. Policy Evaluation/ Prediction: if we have an MDP with the parameters (S, A, P, R , and γ) and a given policy π , then we aim to identify the value function by evaluating the policy using Bellman's Expectation Equations (10.2) and (10.3)
2. Policy Improvement: the value function obtained in the previous step is optimized.
3. Policy Iteration/ Control: From the optimal value function v_* , the optimal policies q_* can be easily obtained through Bellman's Optimality Equations (10.9 and 10.10)

10.5.1.1 Policy Evaluation or Prediction

The policy evaluation involves the computation of state value function v_π for an arbitrary policy π . Please recall the Bellman equation 10.5 in previous section as:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_\pi(s')] \text{ for all } s \in S$$

which provides a unique value of v_π when either $\gamma < 1$ or all states get terminated eventually. When the environment's properties are completely known, then the above equation becomes a system of $|S|$ simultaneous linear equations of $|S|$ unknowns ($v_\pi(s), s \in S$). An algorithm called *iterative policy evaluation* is used to compute the value function. Here a sequence of approximate value functions $v_0, v_1, v_2, \dots$ is considered, each mapping S^+ to $\mathbb{R}$ (real numbers). Initial approximation v_0 is chosen arbitrarily and in each successive approximation is obtained by Bellman equation as an update rule:

$$\begin{aligned} v_{k+1}(s) &= \mathbb{E}_\pi [R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_k(s')] \end{aligned}$$

Please note that the value of the terminal state should always be considered as 0.

This implies that each successive approximation v_{k+1} is computed from v_k through iterative policy evaluation on each state, s . The old value of s is replaced by the new value obtained from the old values of successor states of s , and the expected immediate rewards, for all the one-step transitions possible for the policy under consideration. This way, each iteration of iterative policy evaluation updates the value of every state once to produce a new approximate value function v_{k+1} .

10.5.1.2 Policy Improvement

Now that we have evaluated $v_\pi(s)$ which gives how good it is to follow the current policy from s , we should also evaluate whether we should change the policy to

deterministically choose an action $a \neq \pi(s)$ to improve the value function. This value can be obtained as:

$$q_{\pi}(s, a) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t = a]$$

$$= \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

If q_{π} is greater than v_{π} then it is better to select a in s and then to follow π thereafter than to follow π all the time and also it can be inferred that it is better to select action a every time when s is encountered and the new policy is better one overall. This is called the *policy improvement theorem* and can be expressed as

$$q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s), \pi \text{ and } \pi' \text{ are any pair of deterministic policies for all } s \in S$$

Then the policy π' must be better or as least as good as π and thus obtain great or equal expected return

$$v_{\pi'}(s) \geq v_{\pi}(s)$$

10.5.1.3 Policy Iteration or Control

Once we obtain improved policy $v_{\pi'}$, we can use the same equation to improve it again and obtain an even better policy $v_{\pi''}$. This way each policy is evaluated and then improved in iterations until it reaches the optimal value function and optimal policy. This is shown diagrammatically in Figure 10.7. As finite MDP consists only a finite number of policies, this process is guaranteed to converge to an optimal policy and optimal value function within a finite number of iterations. This process is called *policy iteration*.

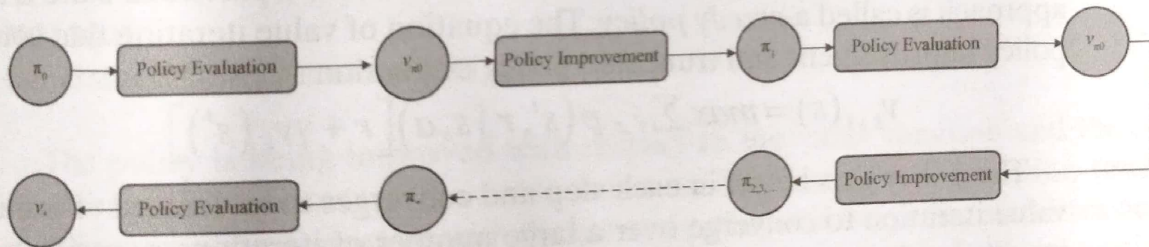


FIG. 10.7
Policy Iteration

The flow of Policy Iteration (iterative policy evaluation) to estimate $\pi \approx \pi^*$ looks like below:

1. Initialize $V(s) \in \mathbb{R}$, $\pi(s) \in A(s)$ for all $s \in S$ arbitrarily, except $V(\text{Terminal})=0$
2. Policy evaluation
 - Loop:
 - diff = 0
 - For each $s \in S$, Loop:
 - $v = V(s)$
 - $V(s) = \sum_{s', r} p(s', r | s, \pi(s)) [r + \gamma V(s')]$
 - diff = max (diff, $|v - V(s)|$)

until $\text{diff} < \epsilon$, a very small number to check whether improvements are happening

3. Policy Improvement

policy-is-stable = True

For each $s \in S$, Loop:

Old-action = $\pi(s)$

$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r | s,a) [r + \gamma V(s')]$$

If old-action $\neq \pi(s)$

policy-is-stable = False

If policy-is-stable then

stop, return $V \approx v_*$ and $\pi \approx \pi_*$

else

move to step 2

10.5.1.4 Value Iteration

It is evident from the policy iteration logic that each of its iterations needs to perform policy evaluation. In the real world, this may involve iterative computation with multiple sweeps through the state set, and for large state sets this can be prohibitively expensive. To optimize the computation requirement without losing the convergence guarantee of policy iteration, the policy evaluation step is truncated in an algorithm called *value iteration*. Here, policy iteration is stopped after just one sweep (one update of each state) because the maximum value for a particular state is taken. This approach is called a *greedy policy*. The equation of value iteration that combines the policy improvement and truncated policy evaluation steps is:

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s',r | s,a) [r + \gamma v_k(s')] \quad (10.10a)$$

The policy gets better in each step and converges to v_* . Instead of waiting for the value iteration to converge over a large number of iterations, normally we stop the iterations when the improvement of the value function becomes very small over each sweep. Please note that equation (10.10) can be directly obtained from the Bellman Optimality equation (10.9) by turning it into an update function.

The complete flow of Value Iteration for estimating $\pi \approx \pi_*$ looks like below:

Initialize $V(s)$ for all $s \in S^+$ arbitrarily, except $V(\text{Terminal}) = 0$
loop for each $s \in S$

1. $\text{diff} = 0$

2. $v = V(s)$

3. $V(s) = \max_a \sum_{s',r} p(s',r | s,a) [r + \gamma V(s')]$

4. $\text{diff} = \max(\text{diff}, |v - V(s)|)$

until $\text{diff} < \Delta$, where Δ is a very small number to check whether improvements are happening

Compute the deterministic policy $\pi \approx \pi_*$ such that

5. $\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r | s,a) [r + \gamma V(s')]$

10.5.1.5 Generalized Policy Iteration (GPI)

We have seen that in Policy Iteration, two simultaneous and interacting processes occur back-to-back – the policy evaluation makes the value function consistent with the current policy and the policy improvement makes the policy greedy with respect to the current value function. We have also seen that in the Value Iteration, efficiency is gained with these two processes being performed simultaneously within a single iteration. So, as long as both processes continue to update all states, the final goal of convergence to optimal value function and optimal policy is achieved.

Generalized Policy Iteration (GPI) is the term that is used to refer to the idea of letting the policy evaluation and policy-improvement processes interact, independent of the granularity of the two processes. Almost all reinforcement learning methods can be described as GPI. The principal concept of GPI is explained in Figure 10.8

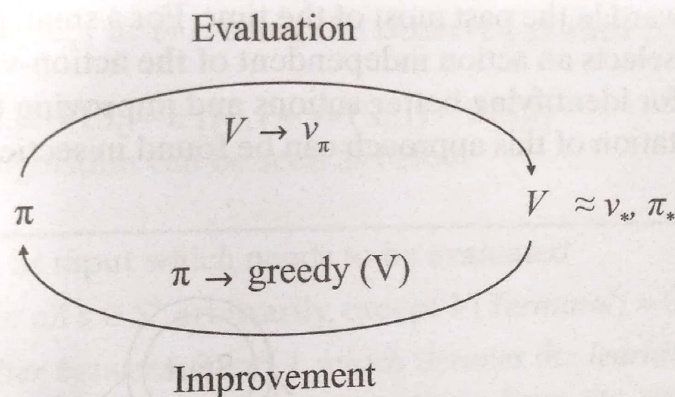


FIG. 10.8
Generalized Policy Iteration (GPI)

The policy is being improved with respect to the value function and the value function is driven toward the value function for the policy. The value function stabilizes once it is consistent with the current policy and the policy stabilizes once it is greedy for the current value function. Thus, the cycle stops once both the processes are optimal. In GPI, the two processes together achieve the optimality goal though neither of them is attempting to achieve it directly.

10.5.2 Monte Carlo Methods

In the previous section, we discussed solving RL problems for MDPs, where we assumed that we have full knowledge of the environment, like the complete probability distribution of all the possible transitions, the reward function, etc. which was the basis for using dynamic programming. But in the real world we may not have the full knowledge of the environment and a new class of RL called model-free learning comes into the picture there. Monte Carlo Methods are one such class of model-free algorithms. In essence, Monte Carlo methods are learning methods that require only *experience* – sample sequences of states, actions, and rewards from actual or simulated interactions with the environment. The methods work by averaging

sample returns. The key assumption is that the experience is divided into episodes and that all episodes terminate at the end irrespective of the actions taken. The value estimates and policies change only after the completion of an episode. The agent takes action and obtains rewards during an episode and we take an average of the rewards when the episode terminates.

$$V(S_t) = V(S_t) + \alpha [G_t - V(S_t)] \quad (10.11)$$

where G_t is actual return following time t and α is a constant step-size parameter

Epsilon-Greedy Action Selection: in section 10.2 we briefly discussed the exploitation vs exploration approach in RL. Extreme dependence either on exploitation or on exploration is detrimental to any model. So, a balance is struck with the use of a small number ϵ (epsilon) to instruct the algorithm when to exploit and when to explore. This is called the Epsilon-Greedy Action Selection approach. Please refer to Figure 10.9. The algorithm exploits, i.e., chooses the optimal action that generated the highest reward in the past most of the time. For a small probability (ϵ) it explores, i.e., randomly selects an action independent of the action-value estimates. This opens up the room for identifying better actions and improving the policy further. The actual implementation of this approach can be found in section 10.6 Python code base.

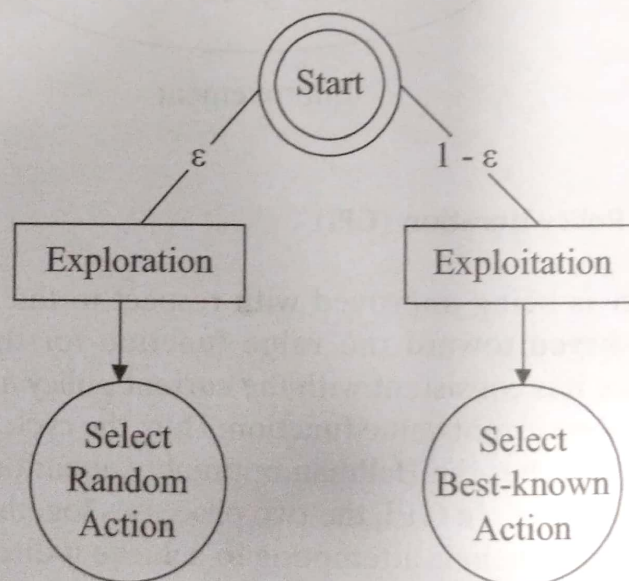


FIG. 10.9

Epsilon-greedy action selection

10.5.3 Temporal Difference Learning (TD)

One of the most effective learning processes in RL is Temporal Difference (TD) learning, which combines the ideas of Monte Carlo methods and DP methods. Similar to Monte Carlo methods, TD methods do not depend on the knowledge of the environment's dynamics or the model and thus learn directly from the experience. Also, similar to DP, TD methods update the estimates based in part on other learned

estimates and do not wait for the final outcome. The steps include Prediction and Control which is a standardized approach for RL algorithms. Please note that DP, Monte Carlo, and TD all use some variation of GPI for the control problem, which aims to find out the optimal policy, but these methods are different in their approaches to the prediction problem, which aims to estimate the value function v_π for a given policy π .

10.5.3.1 TD Prediction

Similar to Monte Carlo methods, TD methods use experience to solve prediction problems. But the main difference in approach is that the Monte Carlo methods wait until an episode completes and the return is known, and then use that return to update the estimate for $V(S_t)$ [refer to equation 10.11]; whereas, TD methods need to wait only until the next time step. The update is done immediately after transitioning to S_{t+1} and receiving R_{t+1} at $t+1$, using the observed reward R_{t+1} and the estimate $V(S_{t+1})$:

$$V(S_t) = V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (10.12)$$

The flow of the algorithm can be seen as below:

Policy π is given as input which needs to be evaluated

Initialize $V(s)$ for all $s \in S^+$ arbitrarily, except $V(Terminal) = 0$, step size $\alpha \in (0,1]$

{ α is a real number between 0 and 1, which denotes the learning rate or step size.

If this is set to zero, then the agent learns nothing from the new actions, whereas making it 1 means that the agent will completely ignore prior knowledge and use only the most recent information. The higher the value of α , faster the changes in value function.}

For each episode, Loop:

Initialize S

For each step of the episode, loop:

1. A = action given by π for S
2. Take action A, observe R, S'
3. $V(S_t) = V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$
4. S = S'

until S is terminal

TD methods use the best parts of both Monte Carlo and DP methods and thus offer advantages over both of them. The advantage over DP methods is that they do not need a model of the environment – its next-state transition probability distribution and the reward. The advantage over Monte Carlo methods is that TD can be implemented in a fully online incremental way. The reason is that Monte Carlo methods need to wait for an episode to end while TD needs to wait only till the next time step. In real-life situations, some of the problems may have very long

running episodes which delay the learning significantly making the process slower. If the constant step-size parameter is kept sufficiently small then TD methods converge to v_π very efficiently.

There are two different approaches to implementing TD prediction methods – on-policy and off-policy, as we will discuss in the subsequent chapters. In on-policy methods, the value function ($Q(s, a)$) is learned by following the current policy $\pi(s|a)$ based on which the actions are chosen. But in off-policy methods, the actions need not follow the current policy, but a different policy can be followed and can even be chosen at random (through exploration). Both these types of methods find an optimal policy in the long run.

10.5.3.2 On-Policy TD Control – SARSA

Here we start by learning the action-value function rather than learning the state-value function. The aim is to estimate $q_\pi(s, a)$ for the current behavior policy π for all state s and actions a . So, instead of learning just the values of the states by transitioning from state to state, we will learn the values of the state-action pairs transitioning from state-action pair to state-action pair. The TD prediction equation of (10.12) will get updated to:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (10.13)$$

The update is made after each transition from non-terminal states. If S_{t+1} is terminal then $Q(S_{t+1}, A_{t+1})$ is defined as zero. You can see that the above equation uses every element of the group of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ [(State-Action-Reward-State-Action)] which has driven the algorithm's name - SARSA. The flow of the SARSA control algorithm is given below, where the main idea is that we continuously estimate q_π for the behavior policy π and at the same time change π to greediness with respect to q_π to finally arrive at the optimum q^* .

Policy π is given as input which needs to be evaluated
 Initialize $Q(s, a)$ for all $s \in S^+$, $a \in A(s)$ arbitrarily, except $Q(\text{Terminal}) = 0$, step size $\alpha \in (0, 1]$, small $\epsilon > 0$
 For each episode, Loop:
 Initialize S
 Choose A from S using policy derived from Q (e.g. ϵ -greedy)
 For each step of the episode, loop:
 1. Take action A , observe R, S'
 2. Choose A' from S' using policy derived from Q (e.g. ϵ -greedy)
 3. $Q(S_t, A_t) = Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
 4. $S = S', A = A'$
 until S is terminal

The algorithm stores a table of state (S)-action (A) estimate pairs for each Q -value. This table is known as a Q -table.

10.5.3.3 Off-Policy TD Control – Q-learning

In 1989 Chris Watkins developed an off-policy TD control algorithm called Q-learning which significantly simplified the algorithm and enabled early convergence proofs. The driving equation is:

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (10.13)$$

As it is evident from the equation, the optimal action-value function q^* is obtained directly from the learned action-value function Q , independent of the policy being followed. The policy only plays a role in determining which state-action pairs are visited and updated, but as long as it is ensured that all pairs continue to be updated, the algorithm converges correctly. This is a model-free learning process, which means that the agent explores the environment and learns directly from it, without constructing an internal model or MDP. The Agent only knows about the possible states and actions in an environment at the beginning and then discovers the state transitions and rewards by exploration.

The sample algorithm of Q-learning is shown below. Python implementation of this algorithm can be found in section 10.6.

Initialize parameters: step size $\alpha \in (0,1]$, small $\epsilon > 0$

Initialize $Q(s, a)$ for all $s \in S^+$, $a \in A(s)$, arbitrarily, except $Q(\text{Terminal}, \cdot) = 0$

For each episode, loop

Initialize S

For each step of the episode, loop:

Choose A from S using policy derived from Q , with the ϵ -greedy approach

Take action A , observe R, S'

$Q(S, A) = Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S = S'$

Until S is terminal

10.6 SAMPLE PYTHON IMPLEMENTATION OF Q-LEARNING

```
# This program implements a sample reinforcement learning algo-
rithm with
# q-learning. The agent learns to reach the end of the frozen lake
through
# trial and error. Note the features like balancing of exploration
and #exploitation, use of value function

import numpy as np
import matplotlib.pyplot as plt
import gymnasium

# defining an agent that learns with q-learning
```



```

class GenericAgent:
    def __init__(
        self,
        learning_rate:float,
        initial_epsilon: float,
        epsilon_decay: float,
        final_epsilon:float,
        discount_factor:float = 0.95
    ):
        self.q_values = np.zeros((env.observation_space.n, env.
        action_space.n))
        self.lr = learning_rate
        self.discount_factor = discount_factor
        self.epsilon = initial_epsilon
        self.epsilon_decay = epsilon_decay
        self.final_epsilon = final_epsilon

        # this function returns a random action (exploration) or the best
        known #action(exploitation) based on the epsilon value
        def get_action(self, obs:tuple[int, int, bool])>int:
            if np.random.random() < self.epsilon:
                return env.action_space.sample()
            else:
                return int(np.argmax(self.q_values[obs,:]))

        # this function calculates the q-value and updates the q-value
        table for #reference while calculating the total cumulative reward
        of a state-action
        def update(self,
            obs:tuple[int, int, bool],
            action:int,
            reward:float,
            terminated:bool,
            next_obs:tuple[int, int, bool]):
            future_q_value = (not terminated * np.max(self.q_values[next_
            obs,:]))
            temporal_difference = (reward + self.discount_factor*future_q_
            value - self.q_values[obs, action])
            self.q_values[obs, action] = (self.q_values[obs, action] +
            self.lr * temporal_difference)

            def decay_epsilon(self):
                self.epsilon = max(self.final_epsilon, self.epsilon
                - epsilon_decay)

```



```

## actual algorithm starts here
learning_rate = 0.01
n_episodes = 10000 # number of times the learning
sequence will run

start_epsilon = 1.0 # at start it goes for exploration and then
moves toward
# exploitation through epsilon decay function
epsilon_decay = start_epsilon/(n_episodes/2)
final_epsilon = 0.1

# initiation of openAI.gymnasium's Frozenlake environment
env = gymnasium.make('FrozenLake-v1', render_mode="rgb_array")
agent = GenericAgent(learning_rate, start_epsilon, epsilon_decay,
                    final_epsilon)

episode_reward = []

for episode in range(n_episodes):
    obs, info = env.reset()
    done = False
    total_episode_reward = 0
    terminated = False

    while not done:
        action = agent.get_action(obs) #either explores or exploits
        # step function returns next observation space, reward value
        of current
        # action and a signal whether the termination action is reached,
        like
        # it has fallen into the lake
        next_obs, reward, terminated, truncated, info = env.step(action)
        agent.update(obs, action, reward, terminated, next_obs)
        frame = env.render()

        plt.imshow(frame)
        plt.show()
        done = terminated or truncated
        obs = next_obs
        total_episode_reward = total_episode_reward + reward
        episode_reward.append( total_episode_reward)
        agent.decay_epsilon()

    for i in range(10):
        print ("Mean reward of episode {} is {}".format((i+1)*1000,
np.mean(episode_reward[i*1000:(i+1)*1000])))

```


The mean reward of 1000 episodes will look like below.

```
>> Mean reward of episode 1000 is 0.026
>> Mean reward of episode 2000 is 0.027
>> Mean reward of episode 3000 is 0.044
>> Mean reward of episode 4000 is 0.053
>> Mean reward of episode 5000 is 0.081
>> Mean reward of episode 6000 is 0.093
>> Mean reward of episode 7000 is 0.092
>> Mean reward of episode 8000 is 0.077
>> Mean reward of episode 9000 is 0.091
>> Mean reward of episode 10000 is 0.096
```

Note that the mean reward value started with a low value as the agent was still learning, but steadily increased as it could maximize the value function much better and use the right policy based on the experience it gained over multiple runs. Can you think of few practical applications of the above algorithm? We have seen that the Agent has learnt to avoid the obstacles on its own and reached the final destination discovering the shortest path. Similar agents like, a self-driving car, an autonomous robot, a computer-player in a game can use this algorithm to continuously learn the policies in the environment and fine-tune their actions.

10.7 APPLICATIONS OF RL

The use of machine learning in solving real-life problems is not new. But, with the advancement in the field of reinforcement learning, the agents now have more capability to recommend or take actions, based on the long-term gain/loss from such actions. In many real-life and industry situations, the volume of data produced is very high, making it difficult to analyze it and create training data from it by manually labeling it. Also, some tasks are dangerous and can't be accomplished by humans. Thus, it is difficult for anyone to train the agent in those situations. The capability of reinforcement learning agents to learn on their own in such situations has made it a popular choice of machine learning algorithm.

Robotics and Industrial Automation

Autonomous robots and industrial machines that can create an adaptive control to learn from their actions and refine future behavior are important examples of the use of RL. Robots and autonomous parts are used widely in applications, like, holding and handling parts of different shapes and sizes, assembling end products, inspecting products for defects, tracking and managing inventory, traveling short and long distances, delivering goods, etc.

Case study: Google's Deepmind has used RL powered agent to manage the cooling systems of Google Data Centres. The workflow that the agent follows is:

1. Measure the environmental conditions within the data center every 5 min
2. Analyse different settings and how those will affect future energy consumption

3. Use sets of standard safety criteria as the guiding policy and identify the actions that will have minimal power consumption
4. Sends and implements the actions

The outcome is very impressive – this led to a 40% reduction in energy spend and thus is a huge step towards sustainability.

Case study: manipulating the complex grasping mechanism for robotic arms was a challenging task when it was attempted to be implemented using only static learning models and vision-based controls. Instead, an RL-based model was developed in 2018, called QT-Opt, that continuously updated the grasping strategy based on the most recent observations and optimized it for long-term grasping success. Application of this RL-based model on real robots by Google AI found that the success rate of grasping increased from 78% to 96% which is very significant in this domain.

Natural Language Processing (NLP)

Text summarization, question-answer-based conversation, and machine translation are a few examples where RL is used in NLP. For example, during machine translation, standard supervised learning is used to predict the words, but the system learns when to use the predicted word or when to wait for more input using the RL model. RL in NLP is now widely implemented by customer service departments in many major organizations.

ChatGPT is a very pertinent example of how language models can be fine-tuned using RL. A language model can generate impressive texts in response to human prompts. But understanding whether the generated text is 'good' in the context of the conversation, was a long-standing challenge. In the case of ChatGPT, human feedback is used as the measure for this performance and it is called reinforcement learning from human feedback (RLHF). The important attributes of the sequence of conversation, like, coherence, informativity, and ease of answering are rewarded in this learning model.

Healthcare

The healthcare sector was one of the early adopters of machine learning. Diagnostic report analysis, medical diagnosis, and ailment prediction have been using machine learning models for quite some time now. With the use of RL, the concept of dynamic treatment regimes (DTRs) has evolved. Based on diagnostic reports, clinical assessment of the patient, and the previous outcomes the learning agent can produce output with suggested treatment type, drug dosages, and appointment timings during a patient's treatment. The result is very encouraging and can reduce the time, energy, and effort in creating an end-to-end treatment strategy.

Gaming

Due to an RL agent's ability to learn and fine-tune its actions through continuous learning, gaming is an area where it is used very extensively. The rules of the game form the policy. In the simulated game environment, the agent navigates, attacks, defends, and strategizes, with a final goal of accumulating a target reward or winning the game.

Did you know?

AlphaGo Zero was able to learn the game of Go from scratch by using reinforcement learning. For 40 days it played against itself. During this, a neural network generated two outputs – a continuous value of the board's state $[-1, +1]$ from the perspective of the current player and a policy which is a probability vector over all possible actions. The estimate of this policy for a given state is improved using Monte Carlo tree search (a reinforcement learning algorithm). After 40 days of self-training, Alpha Go Zero was able to outperform the previous version of AlphaGo known as Master which had defeated world number one Ke Jie.

Self-driving cars

Various aspects of self-driving cars, like, learning speed limits of various places, and avoiding collision, are controlled by deep reinforcement learning. As the autonomous car is supposed to work in real environments, it is not possible to train it for all the possible situations that the car (agent) may encounter on the road. So, the agent learns on its own to identify the best actions, based on the predicted future outcomes. AWS Deep Racer is an autonomous car that is designed to test model self-driving cars in physical tracks. The input from the cameras fitted to the car is used by RL to control the throttle and direction.

Other than the above examples, many other sectors are using RL successfully, like, the finance sector for evaluating trading strategy, recommendation systems in media and e-commerce platforms, pushing targeted marketing and advertisement content based on success rate, etc. The industries are yet to explore the full potential of RL and there will be more widespread interest in this field in the future!

10.8 SUMMARY

- Reinforcement learning is a type of machine learning where the learner learns to map the actions to a situation, to maximize a numerical reward value. The computational approach to learning from interactions with the environment and analyzing the response to generate maximum reward is the basis for Reinforcement Learning.
- An agent makes decisions on what action to take in an environment and in a particular state.
- Everything that the agent interacts with is called an environment.
- The state is the representation of the environment that an agent is in at a specific time step. It is defined as a set of information that the agent has at a given point in time.
- Policy is a strategy that the agent uses to achieve the goal. So, a policy defines the mapping from the perceived states of the environment to actions to be taken when in those states.

- After each action step, the environment sends a single number to the agent, which is termed a reward. These rewards define the goal of the reinforcement learning program because reinforcement learning algorithms have the goal of maximizing the rewards that can be accumulated by the agent over the long run.
- A value function defines long-term gain. The sum of all the rewards that can be accumulated by the agent starting from the current state to all the probable future states is termed as the value of the current state.
- An RL problem is said to have Markov property if the future state depends only on the present and not on the past.
- Rewards vs Returns: Rewards are the numerical values an agent receives based on the action taken while being in a state in the environment. Return is the cumulative reward that the agent receives from the environment while completing a sequence of state transitions, called episodes.
- Discount factor (γ) is the value that determines how much importance is given to the current and future rewards.
- Bellman's equation helps in finding out the optimal policies and value functions to maximize the return.
- A set of algorithms that identify the optimal policy in an environment are
 - & Dynamic Programming (DP)
 - & Monte Carlo Methods
 - & Temporal Difference Learning
 - On-Policy TD Control – SARSA
 - Off-Policy TD Control – Q-learning
- Almost all the industry sectors have started using reinforcement learning extensively, especially for its ability to learn on its own.

SAMPLE QUESTIONS

MULTIPLE-CHOICE QUESTIONS

1. Reinforcement learning methods learned through ____?
 - (a) Experience
 - (b) Predictions
 - (c) Analyzing the data
2. What is an agent in reinforcement learning?
 - (a) Agent is the situation in which rewards are being exchanged
 - (b) Agent is the simple value in reinforcement learning.
 - (c) An agent is an entity that explores the environment.
3. What are actions in reinforcement learning?
 - (a) Actions are the moves that the agent takes inside the environment.
 - (b) Actions are the functions that the environment takes.
 - (c) Actions are the feedback that an agent provides.

4. Which of the following is the practical example of reinforcement learning?
(a) House pricing prediction (b) Market basket analysis
(c) Text classification (d) Driverless cars
5. What are the Rewards of Reinforcement learning?
(a) An agent's action is evaluated based on feedback returned from the environment.
(b) The environment gives value in return which is known as a reward.
(c) A reward is a result returned by the environment after an agent takes an action.
6. In which of the following approaches of reinforcement learning, do we find the optimal value function?
(a) Value-based
(b) Policy-based
(c) Model-based.
7. The agent's main objective is to ____ the total number of rewards for good actions?
(a) Experience
(b) Predictions
(c) Analyzing the data
8. Which element in reinforcement learning defines the behavior of the agent?
(a) Policy (b) Reward Signal
(c) Value Function (d) none of the above
9. Which of the following elements of reinforcement learning imitates the behavior of the environment?
(a) Policy (b) Reward Signal
(c) Value Function (d) none of the above
10. Gamma (γ) in the Bellman equation is known as?
(a) Value factor
(b) Discount factor
(c) Environment factor
11. Why do we use MDP in reinforcement learning?
(a) We use MDP to formalize the reinforcement learning problems.
(b) We use MDP to predict reinforcement learning problems.
(c) We use MDP to analyze the reinforcement learning problems.
12. Which of the following algorithms will find the best course of action, based on the agent's current state, without using a model and off-policy reinforcement learning?
(a) Q-learning
(b) Markov property
(c) State-action-reward-state-action
(d) Deep Q neural network

SHORT-ANSWER TYPE QUESTIONS (5 MARKS EACH)

1. Explain Markov Decision Process.
2. Explain the principle of Q-learning with a flow of the algorithm.
3. Explain various practical applications of RL
4. Why reinforcement learning is the same as supervised or unsupervised learning?

LONG-ANSWER TYPE QUESTIONS (10 MARKS EACH)

1. What is reinforcement learning? Explain with an example.
2. Explain key components of reinforcement learning.
3. Explain value-based, model-based, and policy-based reinforcement learning.
4. What is the Bellman equation? How is it used in RL?